RDBMS LAB RECORD

Car Showroom Management System

Labs 4 to 12

Rohini Pandey
Reg. No.: 2503020050

This record follows the same pattern as the EV Charging Station labs: Lab 4 Constraints, Lab 5 DML, Lab 6 SELECT/WHERE/Operators, Lab 7 Joins, Lab 9 Subqueries, Lab 10 Views, Lab 11 Privileges, Lab 12 TCL, Triggers and Stored Procedures.

Numbering note: Your notebook may label subqueries as Lab 8, while the handout labels them Lab 9. Lab 8 is not in the reference files, so it is skipped here. Use the numbering your instructor requires.

Syntax note: All examples use MySQL syntax. Outputs are representative and match the sample records below. If your database holds different records, your output will differ; copy a table only if it matches the data you inserted.

Tables and Sample Data

Tables used (from the project schema): Supplier, Vehicle_Model, Vehicle_Inventory, Vehicle_Delivery, Showroom_Branch, Customer, Sales_Order. Do not recreate them in the later labs. Sales_Order gets one extra column, Total_Amount (in ₹ lakh), added in Lab 4.

Showroom_Branch

Branch_ID	Name	Address	Contact_Number
B01	City Center Branch	MG Road, Jaipur	0141234567
B02	Airport Road Branch	Airport Road, Jaipur	0141765432
B03	Civil Lines Branch	Civil Lines, Jodhpur	0291456789

Vehicle_Model

Model_ID	Model_Name	Fuel_Type	Category
M01	Nexon	Petrol	SUV
M02	Tiago	Petrol	Hatchback
M03	Swift	Petrol	Hatchback
M04	Creta	Diesel	SUV

Vehicle_Inventory

Inventory_ID	Model_ID	Manufacture_Date	Status	Location
I01	M01	2026-01-15	Available	Bay-1
I02	M02	2026-02-10	Sold	Bay-2
I03	M03	2026-03-05	Available	Bay-3
I04	M01	2026-03-20	Reserved	Bay-1

Model M04 (Creta) has no inventory unit. This is used to show outer joins.

Customer

Customer_ID	Branch_ID	Name	Gender	Age	Phone_Number
C01	B01	Aarav Mehta	Male	32	9123456789
C02	B01	Diya Sharma	Female	28	9812345670
C03	B02	Kabir Singh	Male	41	9700011122

Branch B03 has no customers. This is also used to show outer joins.

Sales_Order

Order_ID	Customer_ID	Branch_ID	Inventory_ID	Order_Date	Quantity_Required	Status	Total_Amount
O01	C01	B01	I02	2026-02-12	1	Fulfilled	7.50
O02	C02	B01	I04	2026-03-25	1	Confirmed	12.00
O03	C03	B02	I01	2026-04-02	1	Pending	11.50
O04	C01	B01	I03	2026-04-10	1	Cancelled	6.80

Supplier (S01 Tata Motors Ltd, 1945; S02 Maruti Suzuki India, 1981; S03 Hyundai Motor India, 1996) and **Vehicle_Delivery** (D01 S01 I01; D02 S01 I02; D03 S02 I03; D04 S01 I04) complete the sample data.

LAB 4: SQL CONSTRAINTS

Aim: To study and implement NOT NULL, UNIQUE, CHECK, PRIMARY KEY, FOREIGN KEY and DEFAULT constraints using the Car Showroom Management System.

Theory: A constraint is a rule applied to a column or table to keep the data valid. The database rejects any statement that breaks the rule.

```sql
-- 1. NOT NULL
ALTER TABLE Customer MODIFY Name VARCHAR(100) NOT NULL;

-- 2. UNIQUE
ALTER TABLE Customer ADD CONSTRAINT uq_customer_phone UNIQUE (Phone_Number);

-- 3. CHECK (enforced in MySQL 8.0.16 and later)
ALTER TABLE Customer ADD CONSTRAINT chk_customer_age CHECK (Age >= 18);

-- 4. PRIMARY KEY and FOREIGN KEY in a new demonstration table
CREATE TABLE Test_Drive (
    Drive_ID     VARCHAR(10) PRIMARY KEY,
    Customer_ID  VARCHAR(10) NOT NULL,
    Inventory_ID VARCHAR(10) NOT NULL,
    Drive_Date   DATE NOT NULL,
    Status       VARCHAR(20) DEFAULT 'Booked',
    FOREIGN KEY (Customer_ID)  REFERENCES Customer(Customer_ID),
    FOREIGN KEY (Inventory_ID) REFERENCES Vehicle_Inventory(Inventory_ID)
);

-- 5. DEFAULT on an existing column, and the new column used in later labs
ALTER TABLE Vehicle_Inventory ALTER COLUMN Status SET DEFAULT 'Available';
ALTER TABLE Sales_Order ADD COLUMN Total_Amount DECIMAL(6,2) DEFAULT 0;
```

These statements normally give a success message, not a data table. Write these result lines below each query:

Constraint	Test	Result to write
NOT NULL	Insert a customer with Name as NULL	ERROR 1048: Column ‘Name’ cannot be null
UNIQUE	Insert a customer with phone 9123456789 again	ERROR 1062: Duplicate entry ‘9123456789’ for key ‘uq_customer_phone’
CHECK	Insert a customer with Age 15	ERROR 3819: Check constraint ‘chk_customer_age’ is violated
PRIMARY KEY	Insert a second Test_Drive row with an existing Drive_ID	ERROR 1062: Duplicate entry for key ‘PRIMARY’
FOREIGN KEY	Insert a Sales_Order with Inventory_ID ‘I99’	ERROR 1452: Cannot add or update a child row: a foreign key constraint fails
DEFAULT	Insert a Sales_Order without Total_Amount	Row saved with Total_Amount = 0.00

Result: Studied and implemented NOT NULL, UNIQUE, CHECK, PRIMARY KEY, FOREIGN KEY and DEFAULT constraints.

Note: Test_Drive is only a demonstration table. Run DROP TABLE Test_Drive; if you do not need it. The exact error text can vary by MySQL version.

LAB 5: DML COMMANDS

Aim: To study and implement INSERT, UPDATE, DELETE and UPSERT commands.

Theory: DML (Data Manipulation Language) commands add, change and remove the data stored in tables.

1. INSERT

```
INSERT INTO Vehicle_Model (Model_ID, Model_Name, Fuel_Type, Category) VALUES
('M01', 'Nexon', 'Petrol', 'SUV'),
('M02', 'Tiago', 'Petrol', 'Hatchback'),
('M03', 'Swift', 'Petrol', 'Hatchback'),
('M04', 'Creta', 'Petrol', 'SUV'),
('M05', 'Venue', 'Petrol', 'SUV');
```

Output: 5 rows affected.

Model_ID	Model_Name	Fuel_Type	Category
M01	Nexon	Petrol	SUV
M02	Tiago	Petrol	Hatchback
M03	Swift	Petrol	Hatchback
M04	Creta	Petrol	SUV
M05	Venue	Petrol	SUV

2. UPDATE: Creta was entered with the wrong fuel type.

```
UPDATE Vehicle_Model SET Fuel_Type = 'Diesel' WHERE Model_ID = 'M04';
```

Output: 1 row affected. M04 now shows Diesel.

3. DELETE: Venue was added by mistake.

```
DELETE FROM Vehicle_Model WHERE Model_ID = 'M05';
```

Output: 1 row affected. Remaining rows:

Model_ID	Model_Name	Fuel_Type	Category
M01	Nexon	Petrol	SUV
M02	Tiago	Petrol	Hatchback
M03	Swift	Petrol	Hatchback
M04	Creta	Diesel	SUV

4. DEFAULT in action

```
INSERT INTO Vehicle_Inventory (Inventory_ID, Model_ID, Manufacture_Date, Location)
VALUES ('I09', 'M02', '2026-04-01', 'Bay-9');
```

Output: 1 row affected. Status is saved as ‘Available’ because of the default set in Lab 4. (Delete I09 afterwards if you do not want it.)

5. UPSERT (MySQL ON DUPLICATE KEY UPDATE)

```
INSERT INTO Vehicle_Model (Model_ID, Model_Name, Fuel_Type, Category)
VALUES ('M03', 'Swift', 'Petrol', 'Hatchback')
ON DUPLICATE KEY UPDATE Fuel_Type = 'Petrol';
```

Result: if M03 does not exist, a new row is inserted; if it exists, the existing row is updated. Here M03 exists and nothing changes, so MySQL reports 0 rows affected.

Result: Studied INSERT, UPDATE, DELETE and UPSERT on project tables.

LAB 6: SELECT, WHERE AND SQL OPERATORS

Aim: To study SELECT, WHERE, LIMIT, ordering, aggregate functions and operators using project tables.

Theory: SELECT retrieves records, WHERE filters them, ORDER BY sorts the result and LIMIT restricts the number of rows returned.

1. SELECT all records

```
SELECT * FROM Vehicle_Inventory;
```

Inventory_ID	Model_ID	Manufacture_Date	Status	Location
I01	M01	2026-01-15	Available	Bay-1
I02	M02	2026-02-10	Sold	Bay-2
I03	M03	2026-03-05	Available	Bay-3
I04	M01	2026-03-20	Reserved	Bay-1

2. WHERE clause

```
SELECT * FROM Vehicle_Inventory WHERE Status = 'Available';
```

Inventory_ID	Model_ID	Manufacture_Date	Status	Location
I01	M01	2026-01-15	Available	Bay-1
I03	M03	2026-03-05	Available	Bay-3

3. Comparison operators (>, >=, <, <=, <>)

```
SELECT Order_ID, Total_Amount, Status FROM Sales_Order WHERE Total_Amount > 10;
```

Order_ID	Total_Amount	Status
O02	12.00	Confirmed
O03	11.50	Pending

Other forms: Total_Amount >= 10, Total_Amount < 10, Total_Amount <= 7.50 and Status <> ‘Cancelled’. For example, Status <> ‘Cancelled’ returns O01, O02 and O03.

4. Logical operators AND, OR, NOT

```
SELECT Inventory_ID, Status FROM Vehicle_Inventory WHERE Status = 'Available' AND Model_ID = 'M01';
```

Output: I01 Available.

```
SELECT Inventory_ID, Status FROM Vehicle_Inventory WHERE Status = 'Available' OR Status = 'Reserved';
```

Output: I01 Available, I03 Available, I04 Reserved.

```
SELECT Inventory_ID, Status FROM Vehicle_Inventory WHERE NOT Status = 'Sold';
```

Output: I01 Available, I03 Available, I04 Reserved.

5. BETWEEN, IN and LIKE

```
SELECT Order_ID, Total_Amount FROM Sales_Order WHERE Total_Amount BETWEEN 8 AND 12;
```

Order_ID	Total_Amount
O02	12.00
O03	11.50

```
SELECT Inventory_ID, Status FROM Vehicle_Inventory WHERE Status IN ('Available', 'Reserved');
```

Output: I01 Available, I03 Available, I04 Reserved.

```
SELECT Customer_ID, Name FROM Customer WHERE Name LIKE 'A%';
```

Output: C01 Aarav Mehta.

6. ORDER BY (one column and two columns)

```
SELECT Order_ID, Total_Amount FROM Sales_Order ORDER BY Total_Amount DESC;
```

Order_ID	Total_Amount
O02	12.00
O03	11.50
O01	7.50
O04	6.80

```
SELECT Order_ID, Status FROM Sales_Order ORDER BY Status ASC, Order_ID ASC;
```

Order_ID	Status
O04	Cancelled
O02	Confirmed
O01	Fulfilled
O03	Pending

7. DISTINCT and LIMIT

```
SELECT DISTINCT Status FROM Vehicle_Inventory;
```

Output: Available, Sold, Reserved (each once).

```
SELECT * FROM Vehicle_Inventory LIMIT 2;
```

Output: rows I01 and I02.

8. Aggregate functions

Query	Output
SELECT COUNT(*) AS Total_Orders FROM Sales_Order;	4
SELECT MAX(Total_Amount) AS Maximum_Amount FROM Sales_Order;	12.00
SELECT MIN(Total_Amount) AS Minimum_Amount FROM Sales_Order;	6.80
SELECT AVG(Total_Amount) AS Average_Amount FROM Sales_Order;	9.45
SELECT SUM(Total_Amount) AS Total_Amount_Sum FROM Sales_Order;	37.80
SELECT COUNT(*) AS Total_Branches FROM Showroom_Branch;	3

Note: COUNT(*) counts the rows of the table named in FROM. Branches give 3 and orders give 4.

9. GROUP BY and HAVING

```
SELECT Model_ID, COUNT(*) AS Units
FROM Vehicle_Inventory
GROUP BY Model_ID
HAVING COUNT(*) > 1;
```

Model_ID	Units
M01	2

Result: Studied filtering, sorting, operators and aggregate queries.

LAB 7: SQL JOINS

Aim: To study and implement different SQL JOIN operations using the Car Showroom Management System.

Theory: A JOIN combines rows from two or more tables using related columns. Row order in join output is not guaranteed unless ORDER BY is used.

1. Cartesian Product (CROSS JOIN)

```
SELECT c.Name AS Customer, b.Name AS Branch
FROM Customer c CROSS JOIN Showroom_Branch b;
```

Output: every customer is paired with every branch (3 x 3 = 9 rows).

Customer	Branch
Aarav Mehta	City Center Branch
Aarav Mehta	Airport Road Branch
Aarav Mehta	Civil Lines Branch
Diya Sharma	City Center Branch
Diya Sharma	Airport Road Branch
Diya Sharma	Civil Lines Branch
Kabir Singh	City Center Branch
Kabir Singh	Airport Road Branch
Kabir Singh	Civil Lines Branch

2. INNER JOIN: matching rows only

```
SELECT c.Customer_ID, c.Name, b.Branch_ID, b.Name AS Branch
FROM Customer c INNER JOIN Showroom_Branch b ON c.Branch_ID = b.Branch_ID;
```

Customer_ID	Name	Branch_ID	Branch
C01	Aarav Mehta	B01	City Center Branch
C02	Diya Sharma	B01	City Center Branch
C03	Kabir Singh	B02	Airport Road Branch

3. LEFT OUTER JOIN: all branches, with customers if any

```
SELECT b.Branch_ID, b.Name AS Branch, c.Customer_ID, c.Name AS Customer
FROM Showroom_Branch b LEFT JOIN Customer c ON b.Branch_ID = c.Branch_ID;
```

Branch_ID	Branch	Customer_ID	Customer
B01	City Center Branch	C01	Aarav Mehta
B01	City Center Branch	C02	Diya Sharma
B02	Airport Road Branch	C03	Kabir Singh
B03	Civil Lines Branch	NULL	NULL

4. RIGHT OUTER JOIN: all models, with inventory units if any

```
SELECT vm.Model_ID, vm.Model_Name, vi.Inventory_ID, vi.Status
FROM Vehicle_Inventory vi RIGHT JOIN Vehicle_Model vm ON vi.Model_ID = vm.Model_ID;
```

Model_ID	Model_Name	Inventory_ID	Status
M01	Nexon	I01	Available
M01	Nexon	I04	Reserved
M02	Tiago	I02	Sold
M03	Swift	I03	Available
M04	Creta	NULL	NULL

5. NATURAL JOIN: joins on the same-named column (here Model_ID)

```
SELECT Inventory_ID, Model_Name, Status
FROM Vehicle_Inventory NATURAL JOIN Vehicle_Model;
```

Inventory_ID	Model_Name	Status
I01	Nexon	Available
I02	Tiago	Sold
I03	Swift	Available
I04	Nexon	Reserved

Use NATURAL JOIN only when the two tables share the intended column name and no other column names in common.

6. FULL OUTER JOIN: MySQL does not support it directly, so combine LEFT JOIN and RIGHT JOIN with UNION

```
SELECT b.Branch_ID, b.Name AS Branch, c.Customer_ID, c.Name AS Customer
FROM Showroom_Branch b LEFT JOIN Customer c ON b.Branch_ID = c.Branch_ID
UNION
SELECT b.Branch_ID, b.Name AS Branch, c.Customer_ID, c.Name AS Customer
FROM Showroom_Branch b RIGHT JOIN Customer c ON b.Branch_ID = c.Branch_ID;
```

Output: all matching rows plus unmatched rows from both tables. With the sample data every customer has a branch, so the result has the same 4 rows as the LEFT JOIN above.

7. Multi-table join: orders with customer, branch and model

```
SELECT o.Order_ID, c.Name AS Customer, b.Name AS Branch, vm.Model_Name, o.Status
FROM Sales_Order o
JOIN Customer c ON o.Customer_ID = c.Customer_ID
JOIN Showroom_Branch b ON o.Branch_ID = b.Branch_ID
JOIN Vehicle_Inventory vi ON o.Inventory_ID = vi.Inventory_ID
JOIN Vehicle_Model vm ON vi.Model_ID = vm.Model_ID;
```

Order_ID	Customer	Branch	Model_Name	Status
O01	Aarav Mehta	City Center Branch	Tiago	Fulfilled
O02	Diya Sharma	City Center Branch	Nexon	Confirmed
O03	Kabir Singh	Airport Road Branch	Nexon	Pending
O04	Aarav Mehta	City Center Branch	Swift	Cancelled

Result: Studied CROSS, INNER, LEFT, RIGHT, NATURAL and simulated FULL OUTER joins for combining related project records.

LAB 9: SUBQUERIES

Aim: To implement single-row, multiple-row, correlated, FROM-clause, UPDATE and DELETE subqueries using the Car Showroom Management System.

Theory: A subquery is a query written inside another SQL statement. Its result (one value or a set of values) is used by the outer query.

1. Single-row subquery: orders above the average amount

```
SELECT Order_ID, Total_Amount
FROM Sales_Order
WHERE Total_Amount > (SELECT AVG(Total_Amount) FROM Sales_Order);
```

Average = (7.50 + 12.00 + 11.50 + 6.80) / 4 = 9.45.

Order_ID	Total_Amount
O02	12.00
O03	11.50

2. Order with the maximum amount

```
SELECT Order_ID, Total_Amount
FROM Sales_Order
WHERE Total_Amount = (SELECT MAX(Total_Amount) FROM Sales_Order);
```

Order_ID	Total_Amount
O02	12.00

3. Multiple-row subquery using IN: customers who have placed orders

```
SELECT Customer_ID, Name
FROM Customer
WHERE Customer_ID IN (SELECT Customer_ID FROM Sales_Order);
```

Customer_ID	Name
C01	Aarav Mehta
C02	Diya Sharma
C03	Kabir Singh

4. Subquery with ANY / SOME

```
SELECT Order_ID, Total_Amount
FROM Sales_Order
WHERE Total_Amount > ANY (
    SELECT Total_Amount FROM Sales_Order WHERE Status = 'Fulfilled'
);
```

The inner query returns 7.50. ANY means greater than at least one returned value.

Order_ID	Total_Amount
O02	12.00
O03	11.50

5. Subquery with ALL

```
SELECT Order_ID, Total_Amount
FROM Sales_Order
WHERE Total_Amount > ALL (
    SELECT Total_Amount FROM Sales_Order WHERE Status IN ('Fulfilled', 'Confirmed')
);
```

Output: Empty set (0 rows), because the inner query returns 7.50 and 12.00, and no order is greater than 12.00.

6. Correlated subquery: orders above that customer's own average

```
SELECT o.Order_ID, o.Customer_ID, o.Total_Amount
FROM Sales_Order o
WHERE o.Total_Amount > (
    SELECT AVG(o2.Total_Amount)
    FROM Sales_Order o2
    WHERE o2.Customer_ID = o.Customer_ID
);
```

Order_ID	Customer_ID	Total_Amount
O01	C01	7.50

For C01 the average is (7.50 + 6.80) / 2 = 7.15, so O01 is above it. C02 and C03 have one order each, which equals their own average.

7. NOT IN: models with no inventory unit

```
SELECT Model_ID, Model_Name
FROM Vehicle_Model
WHERE Model_ID NOT IN (SELECT Model_ID FROM Vehicle_Inventory);
```

Model_ID	Model_Name
M04	Creta

8. Subquery in the FROM clause

```
SELECT x.Status, x.Avg_Amount
FROM (
    SELECT Status, AVG(Total_Amount) AS Avg_Amount
    FROM Sales_Order
    GROUP BY Status
) AS x;
```

Status	Avg_Amount
Cancelled	6.800000
Confirmed	12.000000
Fulfilled	7.500000
Pending	11.500000

9. UPDATE with a subquery: reserve vehicles that have a Pending order

```
UPDATE Vehicle_Inventory
SET Status = 'Reserved'
WHERE Inventory_ID IN (
    SELECT Inventory_ID FROM Sales_Order WHERE Status = 'Pending'
);
```

Output message: 1 row affected (I01 becomes Reserved). To restore the sample data afterwards: UPDATE Vehicle_Inventory SET Status = ‘Available’ WHERE Inventory_ID = ‘I01’;

10. DELETE with a subquery: remove cancelled orders

```
DELETE FROM Sales_Order
WHERE Order_ID IN (
    SELECT Order_ID FROM (
        SELECT Order_ID FROM Sales_Order WHERE Status = 'Cancelled'
    ) AS temp
);
```

Output message: 1 row affected. The extra derived table (temp) is needed in MySQL because a table cannot be changed while the same table is read directly in a subquery. Remaining orders: O01, O02, O03. To restore O04: INSERT INTO Sales_Order VALUES (‘O04’, ‘C01’, ‘B01’, ‘I03’, ‘2026-04-10’, 1, ‘Cancelled’, 6.80);

Result: Studied single-row, multiple-row, correlated, FROM-clause, UPDATE and DELETE subqueries.

Note: Run the UPDATE and DELETE only if the change is wanted, or write the query without running it. Later labs assume the original sample data.

LAB 10: VIEWS

Aim: To create, query, modify and drop views.

Theory: A view is a virtual table based on a SELECT query. It stores the query, not a copy of the data.

1. Create a view of available vehicles

```
CREATE VIEW Available_Vehicles AS
SELECT vi.Inventory_ID, vm.Model_Name, vm.Category, vi.Location
FROM Vehicle_Inventory vi
JOIN Vehicle_Model vm ON vi.Model_ID = vm.Model_ID
WHERE vi.Status = 'Available';
```

Output: View Available_Vehicles created successfully.

```
SELECT * FROM Available_Vehicles;
```

Inventory_ID	Model_Name	Category	Location
I01	Nexon	SUV	Bay-1
I03	Swift	Hatchback	Bay-3

2. Create a view joining orders, customers, branches and models

```
CREATE VIEW Order_Summary_View AS
SELECT o.Order_ID, c.Name AS Customer, b.Name AS Branch,
       vm.Model_Name, o.Total_Amount, o.Status
FROM Sales_Order o
JOIN Customer c ON o.Customer_ID = c.Customer_ID
JOIN Showroom_Branch b ON o.Branch_ID = b.Branch_ID
JOIN Vehicle_Inventory vi ON o.Inventory_ID = vi.Inventory_ID
JOIN Vehicle_Model vm ON vi.Model_ID = vm.Model_ID;
```

Output: View Order_Summary_View created successfully.

```
SELECT * FROM Order_Summary_View;
```

Order_ID	Customer	Branch	Model_Name	Total_Amount	Status
O01	Aarav Mehta	City Center Branch	Tiago	7.50	Fulfilled
O02	Diya Sharma	City Center Branch	Nexon	12.00	Confirmed
O03	Kabir Singh	Airport Road Branch	Nexon	11.50	Pending
O04	Aarav Mehta	City Center Branch	Swift	6.80	Cancelled

A view can be filtered like a table:

```
SELECT Order_ID, Customer FROM Order_Summary_View WHERE Total_Amount > 10;
```

Output: O02 Diya Sharma, O03 Kabir Singh.

3. View with an aggregate: stock per model

```
CREATE VIEW Model_Stock AS
SELECT vm.Model_Name, COUNT(vi.Inventory_ID) AS Units
FROM Vehicle_Model vm
LEFT JOIN Vehicle_Inventory vi ON vm.Model_ID = vi.Model_ID
GROUP BY vm.Model_Name;
```

```
SELECT * FROM Model_Stock ORDER BY Model_Name;
```

Model_Name	Units
Creta	0
Nexon	2
Swift	1
Tiago	1

4. Modify a view using CREATE OR REPLACE

```
CREATE OR REPLACE VIEW Available_Vehicles AS
SELECT vi.Inventory_ID, vm.Model_Name, vm.Category, vi.Location
FROM Vehicle_Inventory vi
JOIN Vehicle_Model vm ON vi.Model_ID = vm.Model_ID
WHERE vi.Status = 'Available' AND vm.Category = 'SUV';
```

Output: View definition replaced. It now returns:

Inventory_ID	Model_Name	Category	Location
I01	Nexon	SUV	Bay-1

5. Drop a view

```
DROP VIEW Order_Summary_View;
```

Output: View removed. The underlying tables are not deleted.

Result: Studied creation, querying, modification and deletion of views.

LAB 11: PRIVILEGES AND ACCESS CONTROL

Aim: To apply and revoke database privileges.

Theory: GRANT gives permissions to a user; REVOKE removes them. This controls who can read or change which tables.

1. Create a test user

```
        CREATE USER 'showroom_staff'@'localhost'
        IDENTIFIED BY 'Showroom_123';
```

Output: User showroom_staff created.

2. Grant SELECT privilege

```
        GRANT SELECT ON CarShowroomDB.Vehicle_Inventory
        TO 'showroom_staff'@'localhost';
```

Output: SELECT privilege granted on Vehicle_Inventory.

3. Check the granted privileges

```
        SHOW GRANTS FOR 'showroom_staff'@'localhost';
```

Representative output:

```
GRANT SELECT ON `CarShowroomDB`.`Vehicle_Inventory` TO `showroom_staff`@`localhost`
```

4. Grant multiple privileges

```
        GRANT SELECT, INSERT, UPDATE
        ON CarShowroomDB.Sales_Order
        TO 'showroom_staff'@'localhost';
```

Output: SELECT, INSERT and UPDATE granted on Sales_Order.

5. Revoke UPDATE

```
        REVOKE UPDATE ON CarShowroomDB.Sales_Order
        FROM 'showroom_staff'@'localhost';
```

Output: UPDATE privilege removed on Sales_Order.

6. Revoke SELECT

```
        REVOKE SELECT ON CarShowroomDB.Vehicle_Inventory
        FROM 'showroom_staff'@'localhost';
```

Output: SELECT privilege removed on Vehicle_Inventory.

7. Optional cleanup

```
        DROP USER 'showroom_staff'@'localhost';
```

Output: User removed.

Result: Studied GRANT, REVOKE and SHOW GRANTS.

Note: Run user and privilege commands only with authorization. Replace CarShowroomDB with the name of your database. The exact SHOW GRANTS text can vary by MySQL version.

LAB 12: TCL, TRANSACTIONS, TRIGGERS AND STORED PROCEDURES

Aim: To use COMMIT, ROLLBACK, SAVEPOINT, triggers and stored procedures.

Theory: A transaction is a group of operations treated as one unit. COMMIT saves the changes, ROLLBACK cancels uncommitted changes and SAVEPOINT marks a point inside a transaction. A trigger runs automatically when a table changes. A stored procedure is a saved block of SQL that is run with CALL.

1. COMMIT

```
START TRANSACTION;
UPDATE Vehicle_Inventory SET Status = 'Sold' WHERE Inventory_ID = 'I04';
COMMIT;
```

Output: I04 changed to Sold and the change is saved.

Inventory_ID	Status
I04	Sold

2. ROLLBACK

```
START TRANSACTION;
UPDATE Vehicle_Inventory SET Status = 'Reserved' WHERE Inventory_ID = 'I03';
ROLLBACK;
SELECT Inventory_ID, Status FROM Vehicle_Inventory WHERE Inventory_ID = 'I03';
```

Output, assuming I03 was Available before this transaction:

Inventory_ID	Status
I03	Available

The uncommitted UPDATE was undone.

3. SAVEPOINT

```
START TRANSACTION;
UPDATE Vehicle_Inventory SET Status = 'Reserved' WHERE Inventory_ID = 'I01';
SAVEPOINT sp1;
UPDATE Vehicle_Inventory SET Status = 'Reserved' WHERE Inventory_ID = 'I03';
ROLLBACK TO sp1;
COMMIT;
```

Output: the change to I01 is committed; the change to I03 is undone.

Inventory_ID	Status
I01	Reserved
I03	Available

4. Create a table for trigger logs

```
CREATE TABLE Order_Log (
    Log_ID      INT AUTO_INCREMENT PRIMARY KEY,
    Order_ID    VARCHAR(10),
    Action_Text VARCHAR(50),
    Action_Time DATETIME
);
```

Output: Table Order_Log created successfully.

5. Create a trigger that logs new orders

```
DELIMITER //
CREATE TRIGGER after_order_insert
AFTER INSERT ON Sales_Order
FOR EACH ROW
BEGIN
    INSERT INTO Order_Log (Order_ID, Action_Text, Action_Time)
    VALUES (NEW.Order_ID, 'Order inserted', NOW());
END //
DELIMITER ;
```

Output: Trigger after_order_insert created successfully.

6. Test the trigger

```
INSERT INTO Sales_Order
(Order_ID, Customer_ID, Branch_ID, Inventory_ID, Order_Date, Quantity_Required, Status, Total_Amount)
VALUES ('O05', 'C02', 'B02', 'I03', '2026-05-01', 1, 'Pending', 6.80);

SELECT Order_ID, Action_Text FROM Order_Log;
```

Representative output, assuming O05 is new and the referenced IDs exist:

Order_ID	Action_Text
O05	Order inserted

7. Create a stored procedure

```
DELIMITER //
CREATE PROCEDURE GetOrdersByStatus(IN p_status VARCHAR(20))
BEGIN
    SELECT Order_ID, Customer_ID, Total_Amount
    FROM Sales_Order
    WHERE Status = p_status;
END //
DELIMITER ;
```

Output: Procedure GetOrdersByStatus created successfully.

8. Call the procedure

```
CALL GetOrdersByStatus('Pending');
```

Order_ID	Customer_ID	Total_Amount
O03	C03	11.50
O05	C02	6.80

9. Create a trigger that keeps inventory in step with sales

```
DELIMITER //
CREATE TRIGGER after_order_fulfilled
AFTER UPDATE ON Sales_Order
FOR EACH ROW
BEGIN
    IF NEW.Status = 'Fulfilled' THEN
        UPDATE Vehicle_Inventory
        SET Status = 'Sold'
        WHERE Inventory_ID = NEW.Inventory_ID;
    END IF;
END //
DELIMITER ;
```

Output: Trigger after_order_fulfilled created successfully.

10. Test the trigger

```
UPDATE Sales_Order SET Status = 'Fulfilled' WHERE Order_ID = 'O03';
SELECT Inventory_ID, Status FROM Vehicle_Inventory WHERE Inventory_ID = 'I01';
```

Output: the order O03 is fulfilled, and the trigger marks its vehicle as Sold.

Inventory_ID	Status
I01	Sold

11. Optional cleanup

```
DROP TRIGGER IF EXISTS after_order_insert;
DROP TRIGGER IF EXISTS after_order_fulfilled;
DROP PROCEDURE IF EXISTS GetOrdersByStatus;
```

Output: Triggers and procedure removed. Cleanup is optional unless your instructor asks for it.

Result: Studied TCL commands, transactions, triggers and stored procedures.

Before executing: These are MySQL examples. Match table and column names to your earlier labs and use your own database name. Do not rerun CREATE statements if the objects already exist. Outputs from Lab 12 depend on the earlier steps having been run in order on the sample data.